PERFORMANCE EVALUATION OF DIJKSTRA'S ALGORITHM IN NETWORK ROUTING

21CSC204J - DESIGN & ANALYSIS OF ALGORITHMS

A MINI-PROJECT REPORT

Submitted By

Panchagnula Mridah Shiva Kumar [RA2411003020287]

L Prashanth [RA2411003020299]

Josh Thomas [RA2411003020305]

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM CAMPUS, CHENNAI-600089

APRIL 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"PERFORMANCE EVALUATION OF DIJKSTRA'S ALGORITHM IN NETWORK ROUTING"** is the bonafide work of **"PANCHAGNULA MRIDAH SHIVA KUMAR RA2411003020287, L PRASHANTH RA2411003020299 , JOSH THOMAS RA2411003020305"** of II Year CSE submitted for the course **21CSC204J - DESIGN & ANALYSIS OF ALGORITHMS** for the Academic Year (2025 – 26) Even Semester.

SIGNATURE

Mr. KINGSLY STEPHEN R

Assistant Professor,

Computer Science and Engineering,
SRM Institute of Science and Technology,
Ramapuram Campus, Chennai.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, CHENNAI -600089

DECLARATION

We hereby declare that the entire work contained in this mini project report titled "**PERFORMANCE EVALUATION OF DIJKSTRA'S ALGORITHM IN NETWORK ROUTING"** has been carried out by **"PANCHAGNULA MRIDAH SHIVA KUMAR RA2411003020287, L PRASHANTH RA2411003020299 , JOSH THOMAS RA2411003020305"** at SRM Institute of Science and Technology, Ramapuram Campus, Chennai – 600089.

Place: Chennai

Date:

ABSTRACT

The project titled “Performance Evaluation of Dijkstra’s Algorithm in Network Routing” focuses on analyzing the efficiency and effectiveness of the Dijkstra's Algorithm in determining optimal paths within a network. In modern communication systems, routing plays a crucial role in ensuring that data packets reach their destination through the most efficient path with minimal delay and resource usage.In this project, a network is modeled as a graph where nodes represent routers and edges represent communication links with associated weights such as distance or transmission cost. The algorithm is implemented using Python and tested on networks of varying sizes to evaluate its performance. Key metrics such as execution time and shortest path cost are analyzed.The study demonstrates that Dijkstra’s Algorithm provides accurate and efficient results for networks with non-negative edge weights. However, its limitations in handling negative weights and large-scale networks are also observed.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	iv
	TABLE OF CONTENTS	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1	**INTRODUCTION**	1
	1.1 Overview	1
	1.2 Problem Statement	1
	1.3 Objective of the Project	2
	1.4 Scope of the Project	2
2	**SOFTWARE AND HARDWARE SPECIFICATIONS**	3
	2.1 Hardware Requirements	3
	2.2 Software Requirements	3
3	**PROJECT DESCRIPTION**	4
	3.1 Introduction	4
	3.2 Architecture Diagram	5
	3.3 Algorithm Used	7
	3.4 Advantages of Algorithm used	8
	3.5 Project Description	9
	3.6 Output	9
	3.7 Results	9
	3.8 Conclusion	11
	References	12
	APPENDIX	
	Source Code	13

LIST OF FIGURES

Fig No.	Figure Name	Page No.
3.1	Dijkstra's Algorithm	6
3.2	Architecture Diagram	8
3.3	Execution comparision	10
3.4	Application in Small Network	10
3.5	Demo Network	11
3.6	Scalibility	11

LIST OF ABBREVIATIONS

- SP – Shortest Path
- DA – Dijkstra's Algorithm
- NSP – Network Shortest Path
- CPU – Central Processing Unit
- RAM – Random Access Memory
- OS – Operating System
- IDE – Integrated Development Environment
- GUI – Graphical User Interface
- API – Application Programming Interface
- OSPF – Open Shortest Path First
- IP – Internet Protocol
- LAN – Local Area Network
- WAN – Wide Area Network
- QoS – Quality of Service
- RTT – Round Trip Time
- BW – Bandwidth
- ET – Execution Time
- PC – Path Cost
- V – Number of Vertices
- E – Number of Edges

CHAPTER 1

INTRODUCTION

1.1 Overview

Routing is a fundamental process in computer networks that determines the path through which data travels from a source node to a destination node. With the increasing complexity of modern networks, the need for efficient routing algorithms has become more significant. These algorithms ensure that data is transmitted with minimal delay, reduced congestion, and optimal utilization of network resources.

One of the most widely used shortest path algorithms is Dijkstra's Algorithm. It is a greedy algorithm that computes the shortest path from a single source node to all other nodes in a graph with non-negative weights. Due to its simplicity and efficiency, it is widely used in real-world routing protocols.

This project aims to simulate a network environment and evaluate the performance of Dijkstra's Algorithm under different conditions, providing insights into its efficiency and scalability.

1.2 Problem Statement

In large and complex networks, selecting the most efficient path for data transmission is a challenging task. Poor routing decisions can result in increased latency, packet loss, and inefficient use of network bandwidth. Although various routing algorithms exist, evaluating their performance under different network conditions is essential to determine their effectiveness.

This project addresses the challenge of analyzing how efficiently Dijkstra's Algorithm performs in finding the shortest path in networks of varying sizes. It also aims to study how the algorithm's execution time and accuracy are affected as the number of nodes and edges increases.

1.3 Aim of the Project

The primary aim of this project is to implement and evaluate the performance of Dijkstra's Algorithm in a simulated network environment. The project seeks to analyze how effectively the algorithm computes shortest paths and how its performance varies with network size and complexity.

Additionally, the project aims to provide a clear understanding of routing mechanisms and highlight the importance of efficient path-finding algorithms in network design.

1.4 Scope of the Project

The scope of this project is limited to the simulation of network routing using graph-based representations. The study focuses on evaluating the performance of Dijkstra’s Algorithm in networks with non-negative edge weights.

The project considers small to medium-sized networks and analyzes performance based on execution time and path cost. It does not include real-time traffic conditions or dynamic routing protocols. However, the findings can be extended to understand the working of practical routing protocols such as OSPF, which rely on shortest path algorithms.

CHAPTER 2

SOFTWARE AND HARDWARE SPECIFICATIONS

2.1 Hardware Requirements

The hardware requirements for the successful implementation of this project are minimal and can be fulfilled using a standard computer system. The required components are as follows:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: At least 500 MB of free disk space
- System Type: Laptop or Desktop computer
- Input Devices: Keyboard and Mouse

These specifications are sufficient to run the program smoothly and perform simulations on small to medium-sized networks without any performance issues.

2.2 Software Requirements

The project is developed using commonly available software tools and programming environments. The requirements are listed below:

Programming Language: Python 3.x

Development Environment: Visual Studio Code / PyCharm / Jupyter Notebook

Libraries Used:

NetworkX (for graph creation and manipulation)

Matplotlib (for visualization and plotting results)

Operating System: Windows / Linux / macOS

These software tools provide a flexible and efficient platform for implementing graph-based algorithms and analyzing their performance.

CHAPTER 3

PROJECT DESCRIPTION

3.1 INTRODUCTION

In the field of computer networking, the efficient transmission of data between devices is a fundamental requirement. As networks continue to grow in size and complexity, the need for intelligent routing mechanisms becomes increasingly important. Routing is the process by which data packets are directed from a source node to a destination node through a series of intermediate nodes. The efficiency of this process directly impacts network performance, including factors such as delay, bandwidth utilization, and reliability.

Shortest path algorithms play a significant role in solving routing problems by identifying the most optimal path between nodes. These algorithms are designed to minimize a specific cost parameter, such as distance, time, or transmission delay. Among the various shortest path algorithms, Dijkstra's Algorithm is one of the most widely used and well-established techniques. Its popularity stems from its simplicity, efficiency, and ability to produce accurate results in graphs with non-negative edge weights.

Dijkstra’s Algorithm operates on the principle of a greedy approach, where the node with the smallest tentative distance is selected at each step. The algorithm begins by assigning an initial distance value to every node, with the source node set to zero and all others set to infinity. It then iteratively updates the shortest distance to each neighboring node until the optimal paths to all nodes are determined. This systematic approach ensures that the final results represent the shortest possible paths within the network.

In practical applications, this algorithm is widely used in routing protocols and network management systems. For instance, it forms the basis of link-state routing protocols such as OSPF, where routers maintain a complete map of the network and compute the shortest paths using Dijkstra’s Algorithm. This highlights the real-world importance of understanding and evaluating its performance.

This project focuses on implementing Dijkstra’s Algorithm in a simulated network environment and analyzing its performance under different conditions. By testing the algorithm on networks of varying sizes and complexities, the study aims to evaluate its

efficiency, scalability, and limitations. The results obtained provide valuable insights into the suitability of the algorithm for different types of network scenarios.

3.2 Algorithm Used

The core of this project is the implementation of Dijkstra's Algorithm, which is a widely used method for finding the shortest path between nodes in a weighted graph. The algorithm is particularly suitable for network routing applications where all edge weights are non-negative. It follows a greedy approach, selecting the node with the smallest known distance at each step and expanding the solution iteratively.

Initially, the algorithm assigns a tentative distance value to every node in the network. The distance to the source node is set to zero, while all other nodes are assigned an infinite value, indicating that they are not yet reachable. A set of visited and unvisited nodes is maintained to track the progress of the algorithm.

At each iteration, the algorithm selects the unvisited node with the smallest tentative distance. This node is considered the current node. The algorithm then examines all its neighboring nodes and calculates the distance from the source node to each neighbor through the current node. If this calculated distance is smaller than the previously recorded distance, it is updated accordingly. This process is known as relaxation.

Once all neighboring nodes have been evaluated, the current node is marked as visited, meaning its shortest distance has been finalized and will not be updated further. The algorithm then proceeds to the next unvisited node with the smallest tentative distance. This process continues until all nodes have been visited or the shortest path to the desired destination has been determined.

To improve efficiency, especially for larger networks, the algorithm can be implemented using a priority queue (min-heap). This allows for faster selection of the node with the smallest distance, significantly reducing the overall time complexity. The time complexity of Dijkstra's Algorithm is typically $O((V + E)log\ V)$when implemented with a priority queue, where Vrepresents the number of vertices and Erepresents the number of edges in the graph.

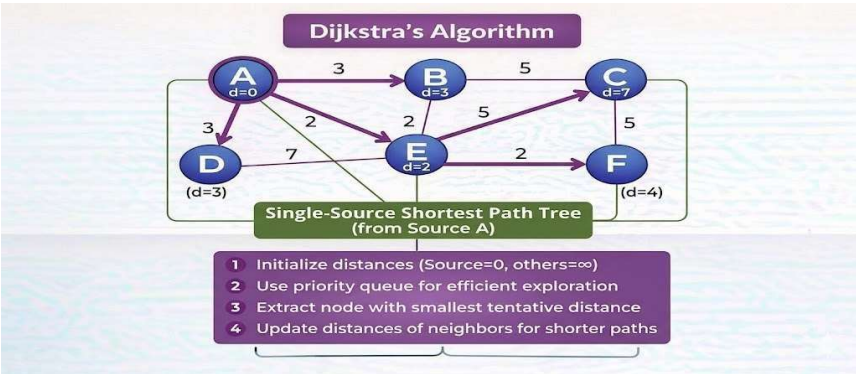

Figure 3.1: Dijkstra's Algorithm

Steps of the Algorithm

- Initialize the distance of the source node to 0 and all other nodes to infinity
- Mark all nodes as unvisited
- Select the node with the smallest tentative distance
- Update the distances of all its neighboring nodes
- Mark the current node as visited
- Repeat the process until all nodes are visited

Pseudocode

```
function Dijkstra(Graph, source):
    create priority queue PQ
    for each vertex v in Graph:
        distance[v] = infinity
    distance[source] = 0
    PQ.push(source, 0)

    while PQ is not empty:
        u = node with minimum distance in PQ
        for each neighbor v of u:
            if distance[u] + weight(u, v) < distance[v]:
                distance[v] = distance[u] + weight(u, v)
                PQ.push(v, distance[v])
```

return distance

This algorithm ensures that the shortest path from the source node to all other nodes is accurately computed. Its efficiency and reliability make it a fundamental component in many real-world networking applications, particularly in routing protocols where optimal path selection is critical.

3.3 Advantages of Algorithm used

The Dijkstra's Algorithm offers several important advantages that make it highly suitable for network routing applications. Due to its deterministic nature and systematic approach, it is widely used in both theoretical and practical implementations of shortest path problems. Its efficiency and accuracy have made it a fundamental algorithm in computer science and networking.

Some of the key advantages of the algorithm are as follows:

- It guarantees the shortest path from the source node to all other nodes in a graph with non-negative edge weights.
- The algorithm is simple and easy to understand, making it suitable for implementation even at a beginner level.
- It provides consistent and reliable results due to its deterministic nature.
- When implemented using efficient data structures such as priority queues, it offers good performance for small to medium-sized networks.
- It is widely used in real-world applications, especially in routing protocols like OSPF, which rely on shortest path computation.
- The algorithm works well for both sparse and moderately dense graphs.
- It can be adapted and optimized further depending on the requirements of the application.

In addition to these advantages, Dijkstra’s Algorithm serves as a foundation for many advanced routing and optimization techniques. Its practical relevance and ease of

implementation make it an essential tool for understanding how efficient routing decisions are made in modern communication networks.

3.4 Architecture Diagram

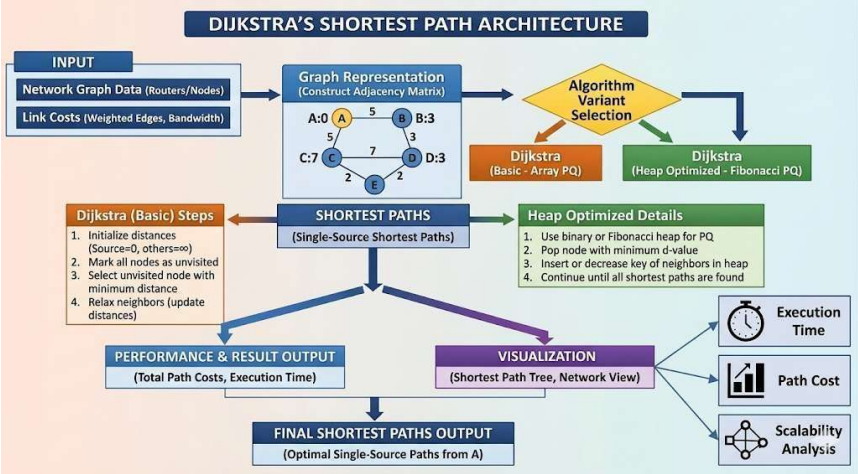

Figure 3.2: Architecture Diagram

The architecture of the proposed system consists of three main components: the input layer, processing layer, and output layer. The input layer is responsible for accepting the network graph, which includes nodes and weighted edges.

The processing layer implements Dijkstra's Algorithm to compute the shortest path from the source node to all other nodes. This layer performs all computations and updates distances dynamically.

Finally, the output layer displays the computed shortest paths along with performance metrics such as execution time.

3.5 Project Description

The project *"Performance Evaluation of Dijkstra's Algorithm in Network Routing"* focuses on analyzing the efficiency of the Dijkstra's Algorithm in determining the shortest path within a network. In this project, a computer network is represented as a graph where nodes act as routers and edges represent connections with associated weights such as distance or delay.

The algorithm is implemented using Python to compute the shortest path from a selected source node to all other nodes in the network. The system evaluates performance by measuring parameters such as execution time and path cost for different network sizes.

The project demonstrates how Dijkstra's Algorithm can be effectively used in network routing to optimize data transmission. It also highlights the strengths and limitations of the algorithm, making it a useful study for understanding routing mechanisms in computer networks.

3.6 Output

The output of the project consists of the shortest path from the selected source node to all other nodes in the network. The results are displayed in a clear and structured format, showing the path and its corresponding cost.

In addition to textual output, the project may also include graphical representations of the network and the computed shortest paths. Performance metrics such as execution time are also displayed to evaluate the efficiency of the algorithm.

3.7 RESULTS

The results obtained from the project indicate that Dijkstra's Algorithm performs efficiently in networks with non-negative edge weights. As the number of nodes increases, the execution time also increases, but the algorithm continues to produce accurate shortest paths.

The performance analysis shows that the algorithm is suitable for small to medium-sized networks. However, for very large networks, the computation time may increase significantly, indicating the need for more optimized or alternative algorithms.

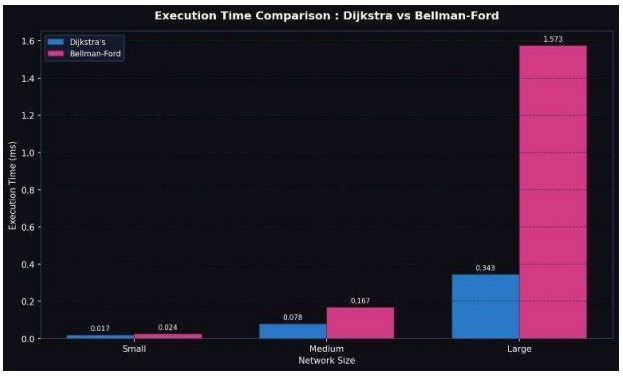

Figure 3.3 Execution Comparision

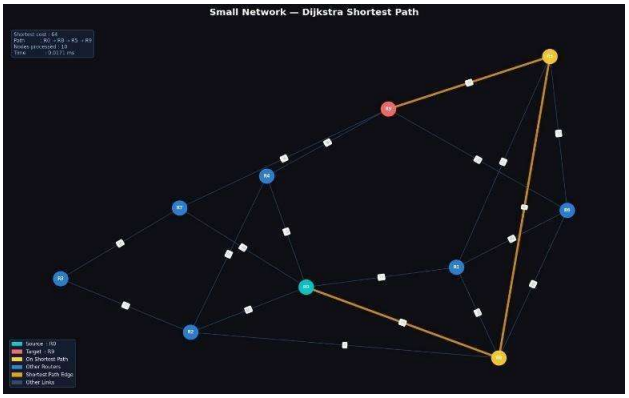

Figure 3.4 Application In Small Network

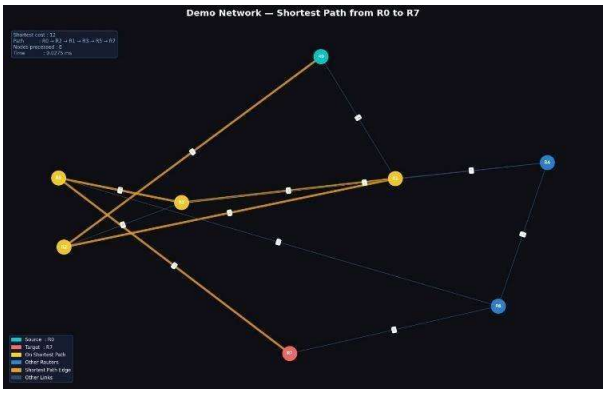

Figure 3.5 Demo Network

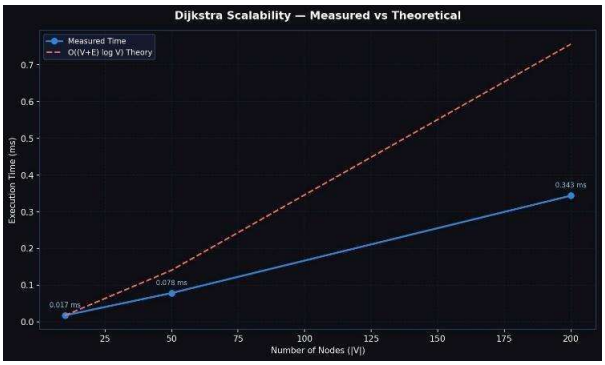

Figure 3.6 Scalability

3.8 Conclusion

The project successfully demonstrates the implementation and performance evaluation of Dijkstra's Algorithm in network routing. The algorithm proves to be efficient, reliable, and easy to implement for computing shortest paths in networks with non-negative weights.

The study highlights the importance of routing algorithms in modern communication systems and provides a clear understanding of how shortest path algorithms contribute to efficient data transmission. While Dijkstra's Algorithm has certain limitations, it remains a fundamental and widely used technique in network routing.

References

[1] T. H. Cormen, *Introduction to Algorithms*

[2] Computer Networking textbooks (Forouzan / Tanenbaum)

[3] Python documentation (official website)

[4] Research papers on shortest path algorithms

[5] Online resources and tutorials related to Dijkstra's Algorithm

APPENDIX

Source Code

```
import heapq

import math

import time


class Graph:


    def __init__(self, num_nodes: int):

        self.num_nodes = num_nodes

        self.adj = {i: [] for i in range(num_nodes)}

        self.edges = []

        self.node_labels = {}


    def add_edge(self, u: int, v: int, weight: float,

                 directed: bool = False):

        if weight < 0:

            raise ValueError(

                "Dijkstra's Algorithm requires non-negative edge weights. "

                f"Edge ({u}→{v}) has weight {weight}."

            )

        self.adj[u].append((v, weight))
```

```
        self.edges.append((u, v, weight))
        if not directed:
            self.adj[v].append((u, weight))

    def set_labels(self, labels: dict):
        self.node_labels = labels

    def __repr__(self):
        lines = [f"Graph({self.num_nodes} nodes, "
                 f"{len(self.edges)} edges)"]
        for u, neighbours in self.adj.items():
            label = self.node_labels.get(u, f"N{u}")
            for v, w in neighbours:
                vlabel = self.node_labels.get(v, f"N{v}")
                lines.append(f"  {label} —[{w}]→ {vlabel}")
        return "\n".join(lines)

def dijkstra(graph: Graph, source: int) -> dict:
    n = graph.num_nodes

    dist = [math.inf] * n
    prev = [-1] * n
    dist[source] = 0
```

```
visited = set()
visited_order = []

heap = [(0, source)]

start_time = time.perf_counter()
nodes_processed = 0

while heap:
    current_dist, u = heapq.heappop(heap)

    if u in visited:
        continue

    visited.add(u)
    visited_order.append(u)
    nodes_processed += 1

    for v, weight in graph.adj[u]:
        if v in visited:
            continue

        new_dist = dist[u] + weight
```

```
            if new_dist < dist[v]:
                dist[v] = new_dist
                prev[v] = u
                heapq.heappush(heap, (new_dist, v))

    execution_time = time.perf_counter() - start_time

    return {
        "distances":        {i: dist[i] for i in range(n)},
        "predecessors":     {i: prev[i] for i in range(n)},
        "visited_order":   visited_order,
        "nodes_processed": nodes_processed,
        "execution_time":  execution_time,
    }

def reconstruct_path(predecessors: dict, source: int,
                     target: int) -> list:
    path = []
    current = target

    while current != -1:
        path.append(current)
        if current == source:
            break
```

```
        current = predecessors[current]
    else:
        return []

    path.reverse()

    if path[0] != source:
        return []

    return path

def bellman_ford(graph: Graph, source: int) -> dict:
    n = graph.num_nodes
    dist = [math.inf] * n
    prev = [-1] * n
    dist[source] = 0

    start_time = time.perf_counter()

    for _ in range(n - 1):
        updated = False
        for u, v, w in graph.edges:
            if dist[u] != math.inf and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
```

```
            prev[v] = u
            updated = True
        if dist[v] != math.inf and dist[v] + w < dist[u]:
            dist[u] = dist[v] + w
            prev[u] = v
            updated = True
    if not updated:
        break

negative_cycle = False
for u, v, w in graph.edges:
    if dist[u] != math.inf and dist[u] + w < dist[v]:
        negative_cycle = True
        break

execution_time = time.perf_counter() - start_time
return {
    "distances":         {i: dist[i] for i in range(n)},
    "predecessors":      {i: prev[i] for i in range(n)},
    "visited_order":     list(range(n)),
    "nodes_processed":  n,
    "execution_time":   execution_time,
    "negative_cycle":   negative_cycle,
}
```